

React + Vite + Tailwind — Cheatsheet

Adivine la palabra — proyecto final LPyL

1. Crear el proyecto

```
npm create vite@latest mi-proyecto -- --template react
cd mi-proyecto
npm install
```

Variante a elegir: **JavaScript** (no TypeScript, no SWC, no React Compiler).

2. Instalar Tailwind

```
npm install tailwindcss @tailwindcss/vite
```

En **vite.config.js**:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

En **src/index.css** (reemplazar todo el contenido):

```
@import "tailwindcss";
```

Con Tailwind v4 ya NO hace falta **tailwind.config.js**. Si copiás ejemplos de Flowbite (hechos para v3) y ves clases como **bg-primary-600** que no funcionan, es porque son colores custom de su config. Reemplazalas por colores estándar (**bg-blue-600**, etc).

3. Correr en desarrollo

```
npm run dev # http://localhost:5173
```

Mientras desarrollás, la app se ve en **localhost:5173** (servidor de Vite), nunca en la URL de Apache. Apache solo sirve la build final.

4. Servir con Apache / XAMPP al terminar

```
npm run build # genera la carpeta dist/
```

Si el proyecto NO está en la raíz de Apache (ej: vive en **/finalJulio/FinalLab/**), hay que indicarle a Vite la ruta base para que los assets se busquen bien:

```
export default defineConfig({
  base: '/finalJulio/FinalLab/dist/',
  plugins: [react(), tailwindcss()],
})
```

Sin esto, Apache busca los JS/CSS en la raíz y la página queda en blanco. El backend PHP va en una carpeta separada, ej: **FinalLab/api/**.

5. Estructura de carpetas

```
src/
  App.jsx          <- decide que pantalla mostrar
  index.css        <- solo el @import de tailwind
  components/
    Login.jsx
    Registro.jsx
    Juego.jsx
    Resultado.jsx
  ui/
```

```
Button.jsx  
Input.jsx  
index.jsx      <- re-exporta todo
```

React — lo básico

Componente y export

```
function MiComponente() {  
  return <h1>Hola</h1>  
}  
export default MiComponente
```

El archivo debe tener extensión **.jsx** (no **.js**) para que Vite procese el HTML mezclado con JS. Y siempre necesita **export default** al final, si no, el import falla con "does not provide an export named default".

Props — pasar datos a un hijo

```
// Hijo  
function Saludo({ nombre }) {  
  return <p>Hola {nombre}</p>  
}  
  
// Padre  
<Saludo nombre="Martín" />
```

Las props también pueden ser funciones, para que el hijo le avise algo al padre (ej: "terminé el login", "cambiá de pantalla").

useState — estado local

```
import { useState } from 'react'  
  
function Contador() {  
  const [cuenta, setCuenta] = useState(0)  
  return (  
    <button onClick={() => setCuenta(c => c + 1)}>  
      Clicks: {cuenta}  
    </button>  
  )  
}
```

Navegación sin router

Para pocas pantallas no hace falta React Router: un solo useState con un string decide qué componente se renderiza.

```
function App() {  
  const [pantalla, setPantalla] = useState('login')  
  
  if (pantalla === 'juego') return <Juego />  
  if (pantalla === 'resultado') return <Resultado />  
  return <Login onLogin={() => setPantalla('juego')} />  
}
```

Formulario controlado

El estado de React, no el DOM, es la fuente de verdad del input. Siempre **value + onChange**, y **e.preventDefault()** en el submit para que no recargue la página.

```
function Login() {  
  const [usuario, setUsuario] = useState('')  
  const [password, setPassword] = useState('')  
  
  const handleSubmit = (e) => {  
    e.preventDefault()  
    console.log(usuario, password)  
  }
```

```

    }

    return (
      <form onSubmit={handleSubmit}>
        <input
          value={usuario}
          onChange={e => setUsuario(e.target.value)}
          required
        />
        <input
          type="password"
          value={password}
          onChange={e => setPassword(e.target.value)}
          required
        />
        <button type="submit">Ingresar</button>
      </form>
    )
  }
}

```

required es validación nativa de HTML, gratis: evita submits vacíos sin escribir código. No reemplaza la validación del backend.

Llamar al backend PHP con fetch

```

const handleSubmit = async (e) => {
  e.preventDefault()
  const res = await fetch('http://localhost/finalJulio/FinalLab/api/login.php', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ usuario, password }),
  })
  const data = await res.json()
  if (data.ok) onLogin(data.usuario)
}

```

Durante el desarrollo, React corre en Vite (5173) y PHP en Apache (localhost) al mismo tiempo. Por eso el fetch usa la URL completa de Apache, no una ruta relativa.

Componentes reutilizables — cuándo sí, cuándo no

Tailwind repite clases. La solución NO es CSS custom, es envolver en un componente React lo que realmente se repite en varias pantallas con la misma pinta.

Sí conviene: Button e Input

Se usan en Login, Registro y Juego. Las clases viven en un solo lugar.

```
function Button({ children, type = 'button', ...props }) {
  return (
    <button
      type={type}
      className="w-full text-white bg-blue-600 hover:bg-blue-700
        font-medium rounded-lg text-sm px-5 py-2.5"
      {...props}
    >
      {children}
    </button>
  )
}
export default Button
function Input({ type = 'text', ...props }) {
  return (
    <input
      type={type}
      className="bg-gray-50 border border-gray-300 text-gray-900
        rounded-lg focus:ring-blue-500 focus:border-blue-500
        block w-full p-2.5"
      {...props}
    />
  )
}
export default Input
```

children es el contenido entre etiquetas de apertura y cierre (ej: el texto del botón). **{...props}** reenvía cualquier otra prop (value, onChange, placeholder, required, etc) al elemento real. no tiene children porque se cierra solo: **<input />**.

Re-exportar varios componentes desde un solo import

```
// src/components/ui/index.jsx
export { default as Button } from './Button'
export { default as Input } from './Input'

// uso:
import { Button, Input } from './ui'
```

No conviene: Label, párrafos, títulos, formularios completos, card de fondo

Son una o dos clases, copiarlas directo en el JSX es más simple que mantener un componente extra. Componente solo cuando el bloque se repite en 3+ lugares Y tiene estructura, no solo una etiqueta suelta.

```
<label className="block mb-2 text-sm font-medium text-gray-900">
  Usuario
</label>
```

Tailwind — referencia rápida

Categoría	Clases
Layout	flex, flex-col, items-center, justify-center, gap-4, grid, grid-cols-2
Espaciado	p-4 (padding 1rem), px-3, py-2, mt-4, space-y-4 (gap entre hijos)
Tipografía	text-sm, text-lg, text-xl, font-semibold, text-center, text-gray-500
Colores	bg-white, bg-gray-50, bg-blue-600, text-white, text-red-500, border-gray-300
Bordes/forma	border, rounded, rounded-lg, rounded-xl, shadow, shadow-md
Estados	hover:bg-blue-700, focus:ring-2, focus:ring-blue-500, disabled:opacity-50

Escala de espaciado: 1 unidad = 4px

1=4px 2=8px 3=12px 4=16px 6=24px 8=32px 10=40px 12=48px

Patrón: pantalla centrada (login, registro)

```
<div className="min-h-screen flex items-center justify-center bg-gray-100">
  <div className="bg-white rounded-xl shadow-md p-8 w-full max-w-sm">
    {/* contenido */}
  </div>
</div>
```

Librerías de componentes ya hechos

Flowbite (flowbite.com) y **daisyUI** (daisyui.com): gratis, copiás el HTML/JSEX directo. **shadcn/ui** (ui.shadcn.com): componentes React + Tailwind, también gratis. Ojo: los ejemplos suelen estar pensados para Tailwind v3 con colores custom — cambiar esos colores por los estándar.

Extensión recomendada de VSCode

Tailwind CSS IntelliSense (Tailwind Labs) — autocompletado de clases y preview de colores.

Errores comunes (ya los pisamos)

Error	Causa / solución
"JSX syntax is disabled"	El archivo tiene extensión .js en vez de .jsx. Renombrar a .jsx.
"does not provide an export named default"	Falta el export default Componente al final del archivo, o el archivo está vacío.
404 en .../dist/src/components/Algo.js	Vite tiene cacheada la ruta vieja. Parar el server (Ctrl+C), npm run dev de nuevo, hard refresh
Pagina en blanco al abrir con Apache	Falta configurar base en vite.config.js apuntando a la subcarpeta real del proyecto.
Clases tailwind tipo bg-primary-600 no hacen nada	Se hace custom de Tailwind v3 (Flowbite). Reemplazar por bg-blue-600 o el color estandar
Error de casing: Button.jsx vs button.jsx	Windows no distingue mayusculas/minusculas en el filesystem pero el import si. Mantener un